

BACKEND PLAN · V2 · NO-FUNDS MODEL

banbe, built without touching the money

The v1 plan assumed banbe collects payment and pays hosts out. That makes you a payment intermediary in Vietnam, with the licensing questions that follow. So take the dodge the product already promises: **the host collects directly, banbe is free, no funds ever sit with you.** This is the same fifteen screens, one layer lighter — and the step-by-step to build it.

1. What changes, exactly

banbe becomes a reservation ledger and a message channel. It records who claimed a seat, tells the host, hands the guest the host's own payment details, and lets the host mark the seat paid. The transfer happens guest → host, in MoMo or a bank app or cash at the door, and banbe never appears in it.

V1 ASSUMED	V2 DOES
MoMo/VNPay integration, create-order and webhook edge functions	Nothing. No wallet SDK, no signature verification, no idempotency worries. The single largest cut.
payments + refunds tables written by webhook	Two columns on bookings: paid_marked_at, paid_method. Host-attested, not authoritative. You are recording a claim, not a transaction.
A 30-minute hold while the guest completes a wallet checkout	The same 30 minutes, new meaning: the window in which the guest sends the transfer and the host confirms it. The countdown UI survives untouched.
Automatic 100% refund on host cancellation	A <i>refund obligation</i> you track and chase: host marks each one returned, guest confirms receipt, banbe shows the timeline and counts failures against the host.
Trust comes from escrow	Trust comes from identity and history: verified phone, attended/no-show counters, host verification badge, and chat as the paper trail.

Say it out loud in the product. The one real cost of this model is ambiguity, and the fix is copy, not code. Every reserve screen needs a line like “*banbe không thu tiền – bạn chuyển trực tiếp cho người tổ chức*”, and every confirmation needs the host’s payment details plus who to contact if it goes wrong. Write those three sentences before step 02; they determine several columns.

2. Stack, settled

Supabase alone: Postgres for the ledger, auth for phone OTP, realtime for chat and the door counter, storage for photos, edge functions for the three or four things that must not run in the browser. Shopify is a product cart and can’t model an expiring seat claim; Square has no Vietnam settlement and, either way, was only ever the payment leg you just removed. Nothing else is needed to launch.

SUPABASE — YOURS

Events, capacity, bookings, check-ins, chat, host profiles and payout details, photos, identity.

HOST — THE MERCHANT

Collecting, receipting, refunding. banbe stores their bank/QR details as display data and never as credentials.

CLIENT — DISPOSABLE

The prototype. Holds no truth; every number on screen is a query result.

3. Schema — twelve tables

Two tables lighter than v1, and one merge worth noting: **the hold is now a booking row in pending**. With no wallet in the loop there is no moment where a claim exists but a booking doesn't, so a separate `seat_holds` table just splits one lifecycle across two places. Prices are integer VND throughout — never floats, never the formatted '900.000đ' strings the prototype carries.

```
-- people -----
profiles      id (=auth.users.id), display_name, phone, phone_verified,
              avatar_url, locale ('vi'|'en'),
              attended_count int, no_show_count int, created_at

organizers    id, owner_id → profiles, name, instagram, about,
              hosting_since, verified bool, verify_requested_at,
              -- collect-direct details, DISPLAY ONLY
              pay_methods text[] ('bank'|'momo'|'cash'),
              bank_name, bank_account_name, bank_account_no,
              momo_phone, pay_qr_path, pay_note,
              refund_pledge text, disputes_open int

-- what's on -----
events        id, organizer_id → organizers, slug, name, category,
              description, included, area, lat, lng, starts_at,
              price_vnd int, capacity int, palette, greeting,
              visibility ('public'|'invite'),
              approval ('instant'|'host_approves'),
              hold_minutes int default 30,
              status ('draft'|'review'|'live'|'cancelled'|'ended'),
              cancelled_at, cancel_reason, created_at

event_photos  id, event_id, storage_path, sort_order
```

```

-- the seat -----
bookings      id, event_id, user_id, qty, total_vnd, code text unique,
               status ('pending'|'confirmed'|'cancelled'|'expired'
                       |'no_show'|'attended'),
               expires_at,                -- pending only
               paid_marked_at, paid_method, paid_marked_by → profiles,
               cancelled_at, cancelled_by, cancel_reason,
               guest_note, created_at, confirmed_at

refund_claims id, booking_id, amount_vnd, reason
               ('host_cancelled'|'guest_cancelled'|'dispute'),
               status ('owed'|'host_marked_sent'|'guest_confirmed'
                       |'disputed'|'waived'),
               host_marked_at, guest_confirmed_at, note

check_ins     booking_id, checked_in_at, checked_in_by → profiles

-- talking -----
threads       id, event_id, guest_id → profiles, organizer_id
messages      id, thread_id, sender_id, body, kind
               ('text'|'system'), created_at, read_at

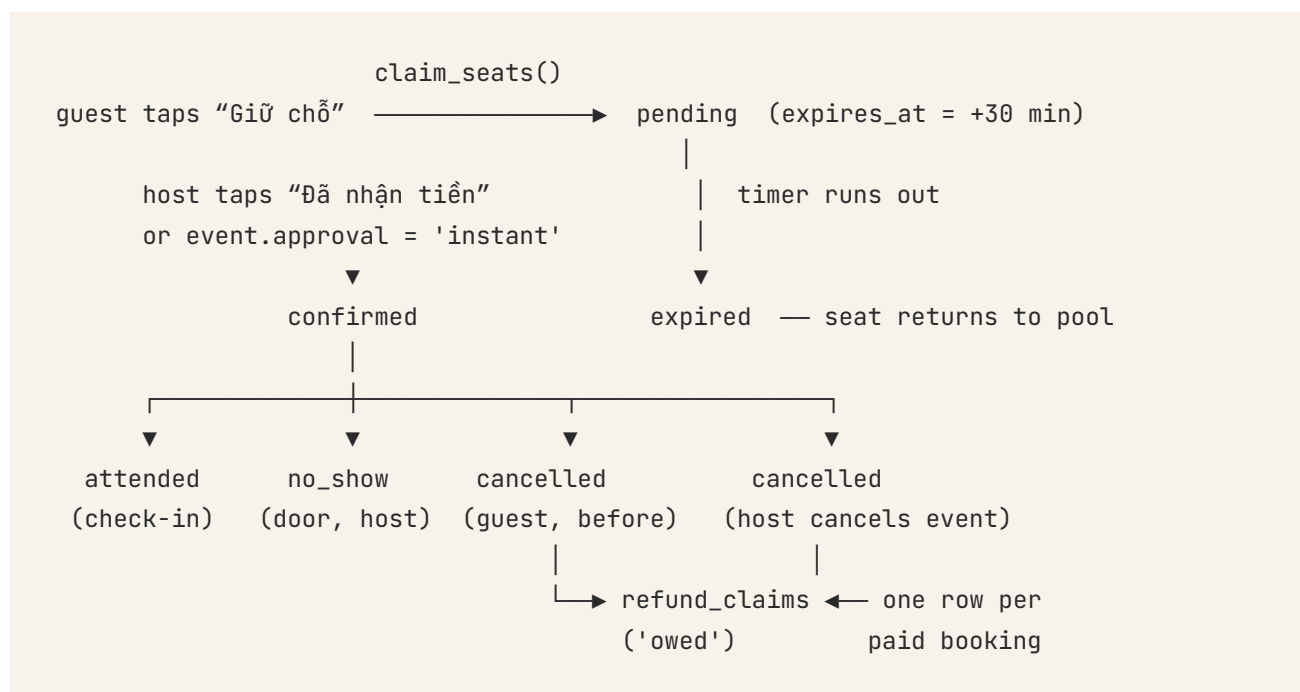
-- soft signals -----
favorites     user_id, event_id
follows       user_id, organizer_id
invites       id, event_id, phone, claimed_by, claimed_at

```

- **Seats stay a count, not rows.** There is no seat map — “Còn 3 chỗ” is capacity minus live claims. Derive it; don’t build a seats table you’ll never read.
- **refund_claims is a chase list, not a ledger.** It exists so a cancelled event produces a visible obligation with a name on it. guest_confirmed is the only status that closes it — the host cannot mark their own refund complete.
- **bookings.code earns its keep.** Six characters, shown to the guest, used as the bank-transfer memo and read out at the door. It’s how the host reconciles a transfer to a seat without you holding funds.
- **Payout fields live on the organizer, not the event** — unless a host tells you otherwise, they collect the same way every time.

4. The booking lifecycle

One state machine, and every screen in the prototype is a view of it.



`approval = 'instant'` is for free and cash-at-door events: the claim confirms immediately and the host chases money at the venue. `'host_approves'` is for prepaid ones and is what the 30-minute countdown belongs to. Let the host choose per event in the create wizard — it's one segmented control and it removes the need for two different products.

5. The one genuinely hard part: not overselling

Two people tapping the last seat in the same second is the bug that embarrasses you in public. You cannot fix it in client code, and you cannot fix it by reading availability then writing — the read is stale before the write lands. It must be one atomic function that locks the event row.

```

create or replace function claim_seats(p_event uuid, p_qty int, p_note text default
null)
returns bookings
language plpgsql security definer as $$
declare v_ev events; v_taken int; v_row bookings;
begin
  -- lock this event so concurrent claims queue behind us
  select * into v_ev from events where id = p_event for update;
  if v_ev.status <> 'live' then raise exception 'NOT_LIVE'; end if;

  select coalesce(sum(qty),0) into v_taken from bookings
  where event_id = p_event
    and (status in ('confirmed','attended')
        or (status = 'pending' and expires_at > now()));

  if v_taken + p_qty > v_ev.capacity then raise exception 'SOLD_OUT'; end if;

  insert into bookings (event_id, user_id, qty, total_vnd, code, guest_note,
                        status, expires_at, confirmed_at)
  values (p_event, auth.uid(), p_qty, p_qty * v_ev.price_vnd,
        upper(substr(md5(gen_random_uuid()::text),1,6)), p_note,
        case when v_ev.approval = 'instant' then 'confirmed' else 'pending' end,
        case when v_ev.approval = 'instant' then null
            else now() + make_interval(mins => v_ev.hold_minutes) end,
        case when v_ev.approval = 'instant' then now() else null end)
  returning * into v_row;

  return v_row;
end $$;

```

The client calls this and nothing else to reserve: `supabase.rpc('claim_seats', {p_event, p_qty})`. It returns the booking — including the code and the countdown deadline the confirmation screen needs — or throws `SOLD_OUT`, which is exactly the sold-out state the event screen already renders. Pair it with a `pg_cron` job, every minute, flipping lapsed pendings to expired; that is what makes the countdown real rather than decorative.

6. Row-level security

Supabase exposes tables straight to the browser, so RLS *is* your API permissions. Enable it on all twelve on day one — an app that works before its policies exist is an app leaking its guest list. In plain language:

- Anyone, signed in or not, reads events that are live and public – the discover feed. Invite-only events resolve by slug only.
- Only the owning organizer writes an event, and only while draft or live.
- A guest reads and cancels their own bookings, never inserts one directly – inserts go through `claim_seats()` so capacity can't be bypassed. Revoke direct insert on bookings and mean it.
- A host reads bookings *for their events only*, and may update just three things: `paid_marked_at`, status along legal transitions, and check-ins. Enforce the transitions in a trigger, not in the app.
- The organizer's payout fields are readable by anyone holding a live booking on one of their events – and by nobody else. Don't put a bank account in the public event payload.
- `refund_claims`: host may set `host_marked_sent`, guest may set `guest_confirmed` or `disputed`. Neither can write the other's field; only service role writes `waived`.
- Messages: readable by the thread's two parties, insertable by either, editable by neither. Counters (`attended_count`, `no_show_count`, `verified`) are trigger-written and client-read-only.

7. Auth, and the thing that will surprise you

Zalo login isn't built in. The login screen offers Zalo, phone, and email. Supabase ships email and phone OTP; Zalo is not one of its providers, so you wire it yourself as a generic OAuth flow through a Zalo Official Account inside an edge function, then mint a Supabase session from the result. Budget real days, and launch on phone OTP. Vietnamese SMS delivery wants a local provider rather than Twilio.

Phone OTP matters more in this model than it did in v1: with no card on file, **a verified phone number is the only thing standing between a host and a stranger who books six seats for fun.** Require it before `claim_seats()` succeeds – one line in the function, checking `profiles.phone_verified`.

8. Screen → data map

SCREEN	READS	WRITES
Login	auth	profiles (trigger)
Home / discover	events + v_event_availability, event_photos, favorites	favorites
Event detail	events, organizers (public fields), photos, follows	follows, favorites
Reserve	events (price, capacity, approval), profiles.phone_verified	claim_seats()
Confirmed / pending	bookings (code, expires_at), organizers payout fields (realtime)	– (host confirms)
Cancelled / refund	refund_claims, bookings, events.cancel_reason	refund_claims.guest_confirmed / disputed
Inbox / chat	threads, messages (realtime)	messages, read_at
Organizer dashboard	organizers, events, seats sold, unpaid pendings, refunds owed	organizers, bookings.paid_marked_at
Check-in	bookings + profiles + code, check_ins (realtime)	check_ins, status → attended / no_show
Create event	organizers (payout completeness)	events (draft), event_photos, storage

9. Build it in this order

Thirteen steps. Each one leaves the app running, nothing needs the step after it, and each has a test that tells you it's finished. Roughly four to six weeks solo; steps 06 and 12 are the long ones.

01 Write the money paragraph

Before any code: three sentences, in Vietnamese, that state banbe doesn't collect, that payment goes to the host, and what happens if a host cancels or ghosts. Put them on the reserve screen, the confirmation, and the host onboarding. Then spend a lawyer's hour confirming that a free, non-custodial reservation tool needs no payment licence – that hour is the entire premise of this plan.

DONE WHEN • THE COPY IS IN THE PROTOTYPE, NOT A DOC

02 Project and tooling

New Supabase project in Singapore – nearest region to Vietnam, and latency is felt on the door scanner. Install the CLI, `supabase init`, `supabase link`, commit the repo. Every schema change from here is a migration file; nothing gets clicked in the dashboard, because you will run all of this again on a staging project.

DONE WHEN • `supabase db reset` REBUILDS FROM ZERO LOCALLY

03 Schema migration

The twelve tables from §3, as enums not free text, with foreign keys and on delete rules decided deliberately. Add the indexes now: `bookings(event_id, status)`, `bookings(user_id)`, `bookings(code)`, `events(status, starts_at)`, `messages(thread_id, created_at)`. Then one view, `v_event_availability`, that returns capacity, live claims, and seats left – the feed queries the view, never recomputes the arithmetic.

DONE WHEN • THE VIEW RETURNS CORRECT SEATS-LEFT FOR HAND-INSERTED ROWS

04 RLS before data

Enable RLS on all twelve tables, write the §6 policies, and revoke direct insert on bookings. Write the transition trigger too – a booking may only move along the arrows in §4, and an illegal update raises rather than silently corrupts. Then test each policy three times: signed out, as a guest, as a host who owns nothing relevant.

DONE WHEN • HOST A'S TOKEN RETURNS ZERO ROWS OF HOST B'S BOOKINGS

05 Storage and seed

Two buckets: `event-photos` public, `pay-qr` private with signed URLs – a host's payment QR is not a public asset. Upload the images in `photos/`, then port the prototype's events, organizers and photo rows into a seed script, converting the formatted price strings to integer VND as you go.

DONE WHEN • A RESET REPRODUCES THE FULL DEMO DATASET IN ONE COMMAND

06 Read path only

Swap the prototype's hardcoded arrays for queries – discover, event detail, organizer page – behind a thin data module so no screen imports the Supabase client directly. No auth, no writes. This is the cheap step where you discover the schema is wrong, so do it before anything depends on the schema.

DONE WHEN • DELETING A ROW IN THE DASHBOARD CHANGES THE FEED ON RELOAD

07 Auth and identity

Phone OTP with a Vietnamese SMS provider, email as fallback, Zalo deferred to step 13. A trigger on `auth.users` creates the `profiles` row; verification sets `phone_verified`. Rate-limit OTP sends per number — SMS costs money and bots find endpoints fast.

DONE WHEN • A REAL PHONE SIGNS IN AND LANDS WITH A PROFILE ROW

08 Claims and expiry — the core

Ship `claim_seats()` from §5 plus the per-minute `pg_cron` expiry job, and wire the reserve screen to the RPC. Add the guard clauses: phone verified, no more than one pending booking per guest per event, a per-event maximum party size. Then test it properly — twenty concurrent claims against a three-seat event, and exactly three win.

DONE WHEN • THE CONCURRENCY TEST NEVER OVERSELLS, AND HOLDS SELF-RELEASE

09 The collect-direct handoff

The substitute for a payment integration, and it's mostly UI. The pending screen shows the host's bank line or MoMo number, the signed QR, the amount, and the booking code as transfer memo — with a copy button on each, because people are standing in a bank app. The host's dashboard gets one row per pending booking and one button, *Đã nhận tiền*, which sets `paid_marked_at` and promotes the booking to confirmed; the guest's screen flips over realtime. Send a system message into the thread on both events so the chat is the receipt trail.

DONE WHEN • TWO DEVICES COMPLETE A CLAIM → CONFIRM CYCLE WITHOUT A REFRESH

10 Cancellation and refund obligations

A trigger on `events.status` → 'cancelled' cancels every live booking and opens a `refund_claims` row for each one already marked paid. The existing three-step refund timeline now reads `owed` → `host_marked_sent` → `guest_confirmed` — same design, honest words: the host is returning your money, here's the button if it doesn't arrive. Unresolved claims after 72 hours increment `disputes_open` and surface on your admin list. This is the model's weak point, so make it the most visible thing you operate.

DONE WHEN • CANCELLING A SEEDED EVENT PRODUCES THE RIGHT CLAIMS AND SCREENS

11 Chat

Threads and messages over a Supabase realtime channel per thread; the inbox unread dot is `read_at is null`. A thread opens automatically on first claim, seeded with the host's greeting from the event. Chat isn't a nice-to-have here — with no escrow it's where payment questions get settled, so it should feel fast and be searchable by you when a dispute lands.

DONE WHEN • MESSAGES ARRIVE ON A SECOND DEVICE IN UNDER A SECOND

12 Host tools

Create-event wizard writing drafts plus photo upload; the payout details step is mandatory before an event can go live, and the approval mode is a segmented control there. Dashboard aggregates: seats sold, unpaid pendings, refunds owed. Check-in over realtime, searchable by name or booking code, with attended and no-show both one tap – two people working the door must not double-tap the same guest.

DONE WHEN • A HOST CREATES, FILLS, AND CHECKS IN AN EVENT WITHOUT YOU

13 Then the nice things

Zalo login, invite links, distance sorting with PostGIS, push notifications, host verification review. None of these block a first cohort.

DONE WHEN • LAUNCHED, AND YOU'RE CHOOSING FROM REAL COMPLAINTS

10. Server code you actually write

The whole backend, itemised. This is the point of the no-funds model: it fits on one page.

postgres functions	claim_seats(event, qty, note) confirm_payment(booking, method) cancel_booking(booking, reason) cancel_event(event, reason) check_in(code booking)	guest reserves host marks received either side opens refund claims door
triggers	profiles row on signup legal booking status transitions attended / no_show counters thread opened on first claim system message on confirm & cancel	
pg_cron	every minute hourly daily	expire lapsed pendings flag refund claims past 72h mark past events 'ended'
edge functions	send-otp signed-qr zalo-oauth	(local SMS provider) (private pay-qr URLs) (step 13)
views	v_event_availability v_host_ledger	capacity, taken, seats_left per event: sold, unpaid, owed

When you do eventually take money. The migration path is deliberately short, and nothing above blocks it: add back the payments and refunds tables, point pending at a wallet checkout instead of a bank memo, and let the webhook call `confirm_payment()` where the host's button called it. The state machine, the capacity lock, the RLS and every screen stay exactly as built.

Do it when one of three things is true: hosts are ghosting refunds, guests won't transfer to strangers, or you want a commission. Until then, custody is a liability you don't need.